

Instituto Tecnológico de Estudios Superiores de Monterrey
Campus Estado de México

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)

Profesores: **Jorge Adolfo Ramírez Uresti**

Elisabetta Crescio, Julio G. Arriaga Blumenkron, Israel Sanchez Miranda,
Víctor Adrián Sosa Hernández, Miguel González Mendoza, Alberto Michel Pérez Domínguez

Momento de Retroalimentación: Módulo 2 Análisis y Reporte sobre el desempeño del modelo. (Portafolio Análisis)

Random Forest

Emilio Páez De la Mora - A01801224

Dataset y separación

Este análisis se realizó sobre la implementación con framework de la parte 2: un Random Forest (scikit-learn) entrenado sobre el mismo dataset de las entregas anteriores, Online Shoppers Purchasing Intention (Sakar & Kastro, 2018, UCI Machine Learning Repository, licencia CC BY 4.0), con las mismas 12 variables y el mismo preprocesamiento manual (Weekend, VisitorRecurrente y Revenue codificados a mano).

A diferencia de las partes 1 y 2, que usaban una separación de dos vías (80% entrenamiento, 20% prueba), este entregable requiere un tercer conjunto: validación. Los tres conjuntos cumplen roles distintos y no se mezclan entre sí:

- Entrenamiento: 60% (7398 registros), sirve para ajustar los árboles del modelo
- Validación: 20% (2466 registros), sirve para diagnosticar bias/varianza y elegir hiperparámetros para grid search
- Prueba: 20% (2466 registros), es nuestra evaluación final única, nunca antes usado para decidir nada, es decir información nueva para el modelo

Se utilizó una semilla fija para la reproducibilidad, extendida a dos cortes en vez de uno como en las anteriores partes. El desbalance de clases se mantiene consistente en los 3 conjuntos: 84.5% no compra, 15.5% sí compra en el total, con porcentajes casi idénticos en cada subconjunto. Es importante notar que este conjunto de prueba no es exactamente el mismo que el de las partes 1 y 2, porque el corte adicional de validación desplaza los índices de la separación aleatoria

Metodología de diagnóstico

El diagnóstico de bias y varianza se basa en comparar F1 en entrenamiento vs F1 en validación. La lógica, implementada en [diagnostico.py](#), sigue el siguiente razonamiento:

- **Bias:** indica el desempeño del entrenamiento. Si el modelo no logra un F1 razonable en los datos que ya vió, es señal de que le falta capacidad de capturar el patrón, es decir tiene un **bias alto**. Se clasifica como bajo si F1 entrenamiento ≥ 0.75 , alto si < 0.45 y medio en el rango intermedio.
- **Varianza:** la indica la brecha entre F1 de entrenamiento y F1 de validación. Una brecha grande significa que el modelo memoriza detalles del entrenamiento que no generalizan. Se clasifica como baja si la brecha ≤ 0.05 , alta si > 0.15 , y media en el rango intermedio.
- **Nivel de ajuste:** se deriva de combinar los dos diagnósticos anteriores. Bias alto se traduce en underfit sin importar la varianza. Con bias bajo o medio, varianza alta se traduce en overfitting; varianza baja o media se traduce en un buen ajuste (fit).

Estos umbrales son un criterio razonable para el problema específico (F1 con clases desbalanceadas, en un rango típico de 0 a 0.7 para este dataset) y se documentan explícitamente para

que el diagnóstico sea reproducible y no quede a juicio subjetivo.

Diagnóstico del modelo base (antes de regularizar)

El modelo base usa la misma configuración de modelo_1 en la parte 2: `n_estimators=100`, `max_depth=None` (sin límite), entrenado únicamente con el conjunto de entrenamiento.

```
2. MODELO BASE (antes de regularizar) - DIAGNOSTICO

Configuracion: n_estimators=100, max_depth=None
Entrenado SOLO con el conjunto de entrenamiento.

Diagnostico ANTES de regularizar
-----
F1 entrenamiento: 0.9991
F1 validacion:    0.6230
Brecha:          +0.3761

Bias:    bajo
Varianza: alta
Ajuste:  overfit
```

El F1 de entrenamiento de 0.9991 confirma un bias bajo: el modelo tiene sobrada capacidad para ajustar los datos que ya vio, casi a la perfección. Sin embargo, el F1 cae a 0.6230 en validación, una brecha de +0.3761 muy por encima del umbral de varianza alta (0.15). El diagnóstico combinado es overfit: el modelo memorizó particularidades del conjunto de entrenamiento (cada árbol sin límite de profundidad llega a hojas puras de muy pocos registros) que no generalizan a datos nuevos. Este es el mismo fenómeno documentado en el experimento de `max_depth` de la parte 2, ahora confirmado formalmente con un conjunto de validación independiente en vez de solo observar la forma de una curva.

Regularización: búsqueda de hiperparámetros

Para corregir el sobreajuste diagnosticado, se usó `GridSearchCV` sobre una rejilla de hiperparámetros, con una diferencia importante respecto a la parte 2: en vez de dejar que `GridSearchCV` parta el entrenamiento en `k` folds automáticamente, se usó un `PredefinedSplit` que combina los conjuntos de entrenamiento y validación en un solo arreglo y le indica explícitamente a `sklearn` cuáles filas son de entrenamiento (excluidas de la evaluación) y cuáles son de validación (usadas para puntuar cada combinación). Esto asegura que la búsqueda respeta exactamente la separación de tres vías del entregable, en vez de mezclar validación con entrenamiento a través de un `k-fold` interno.

La rejilla explorada incluyó cuatro hiperparámetros, dos de ellos técnicas explícitas de regularización adicionales a las ya usadas en la parte 2:

- `n_estimators`: regula el número de árboles del bosque, fue probado con 50, 100 y 200
- `max_depth`: regula la profundidad máxima de cada arboles, fue probado con 5, 8, 10, 15, sin límite
- `min_samples_leaf`: regula el mínimo de muestras por hoja (evita hojas de 1-2 registros), fue probado con 1, 5, 10, 20.
- `class_weight`: compensa el desbalance de clases en el entrenamiento, fue probado con `None` y `'balanced'`

En total se probaron $3 \times 5 \times 4 \times 2 = 120$ combinaciones, cada una evaluada una sola vez (entrenada con el conjunto de entrenamiento, puntuada con F1 sobre el conjunto de validación), en 17.5 segundos

```
3. BUSQUEDA DE HIPERPARAMETROS (GridSearchCV con train/validation)

Combinaciones probadas: 120
Tiempo de busqueda:      17.6 s
Mejor F1 en validacion:  0.6708
Mejores hiperparametros: {'class_weight': 'balanced', 'max_depth': 15, 'min_samples_leaf': 5, 'n_estimators': 100}
```

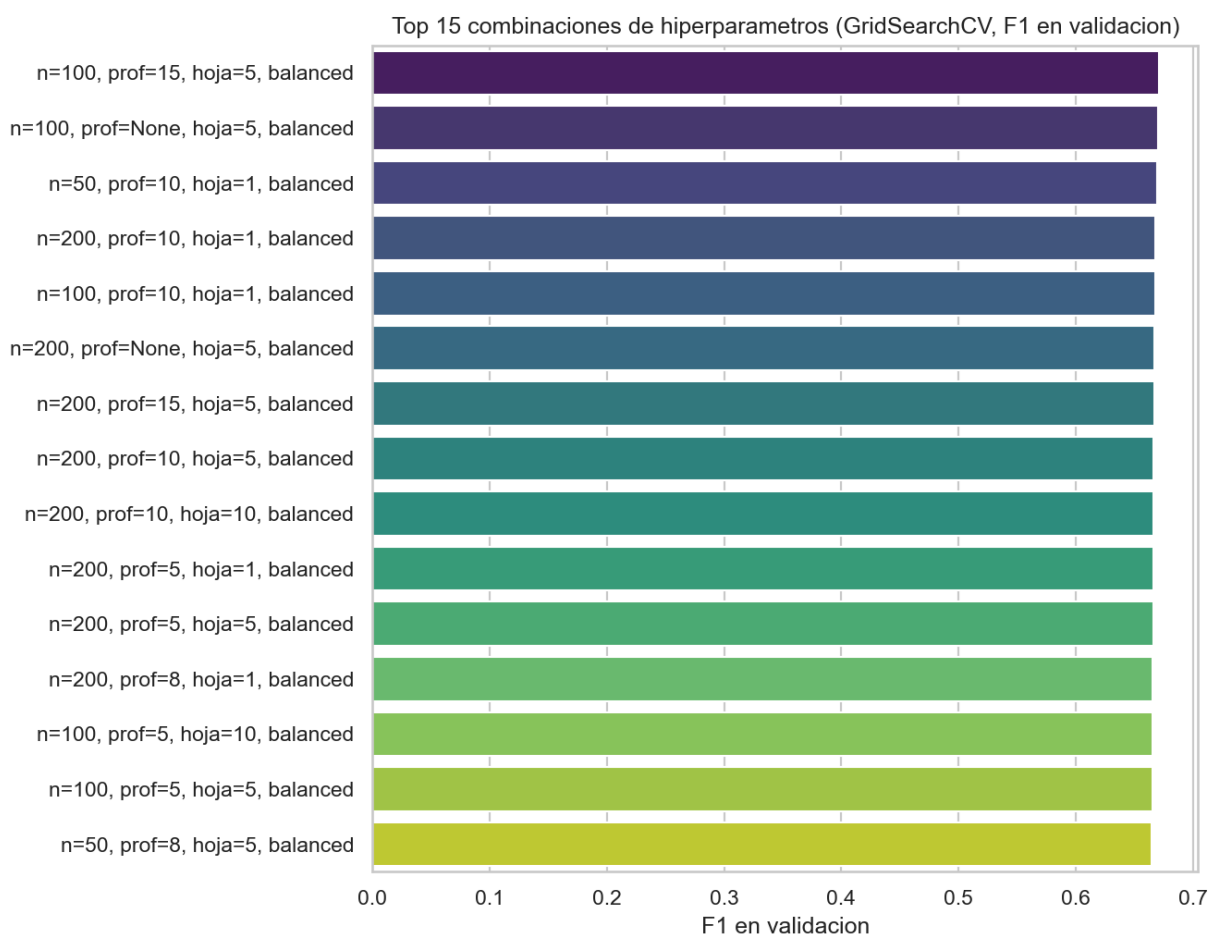


Figura 1. Top 15 combinaciones de hiperparámetros según F1 en validación.

Las 15 mejores combinaciones (Figura 1) tienen F1 muy similar entre sí (todas cercanas a 0.66-0.67) y, salvo una excepción, todas usan `class_weight='balanced'`. Esto es la evidencia más clara de la búsqueda: compensar el desbalance de clases aporta más al F1 de validación que ajustar finamente la profundidad o el número de árboles. La combinación ganadora fue `max_depth=15, min_samples_leaf=5, n_estimators=100, class_weight='balanced'`.

Diagnóstico del modelo regularizado (después)

Con los hiperparámetros encontrados, se entrenó un nuevo modelo, otra vez usando únicamente el conjunto de entrenamiento (los mismos datos que el modelo base), para que el diagnóstico antes/después sea directamente comparable.

4. MODELO REGULARIZADO (despues) - DIAGNOSTICO

```
Configuracion: {'class_weight': 'balanced', 'max_depth': 15, 'min_samples_leaf': 5, 'n_estimators': 100}
Entrenado SOLO con el conjunto de entrenamiento (misma data que el modelo base,
para que la comparacion antes/despues sea justa).
```

Diagnostico DESPUES de regularizar

```
-----
F1 entrenamiento: 0.7623
F1 validacion:    0.6708
Brecha:           +0.0915
```

```
Bias:    bajo
Varianza: media
Ajuste:  fit
```

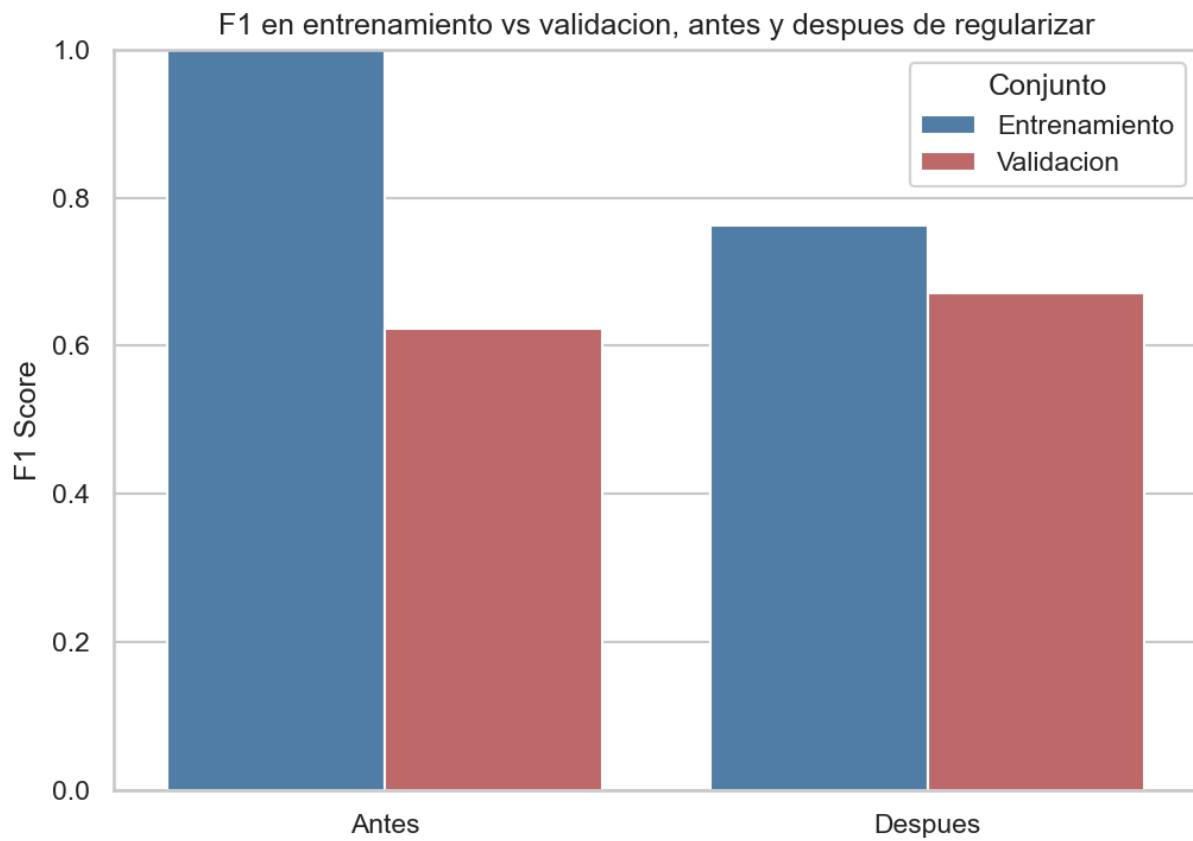


Figura 2. F1 en entrenamiento vs. validación, antes y después de regularizar.

El contraste es claro en la Figura 2: antes, la barra de entrenamiento casi toca 1.0 mientras la de validación se queda muy por debajo; después, ambas barras están mucho más cerca entre sí. Numéricamente, la brecha se redujo de +0.3761 a +0.0915, un 76% de reducción. El bias se mantiene bajo (el modelo sigue teniendo capacidad suficiente para capturar el patrón: 0.7623 de F1 en entrenamiento), pero la varianza bajó de alta a media, y el diagnóstico de ajuste pasó de overfit a fit. Este es precisamente el resultado esperado de limitar `min_samples_leaf` y `max_depth`: se sacrifica algo de F1 en entrenamiento (el modelo ya no puede memorizar hojas de un solo registro) a cambio de que ese aprendizaje generalice mejor a datos nuevos.

Evaluación final sobre el conjunto de prueba

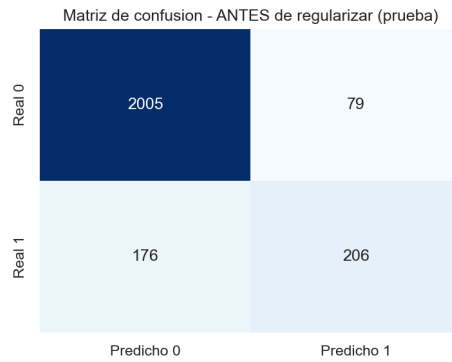


Figura 3. Matriz de confusión — modelo base, conjunto de prueba.

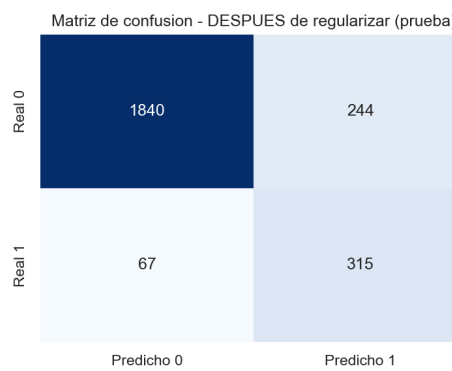


Figura 4. Matriz de confusión — modelo regularizado, conjunto de prueba.

Mejora en el conjunto de prueba:

accuracy	0.8966	->	0.8739	(-0.0227)
precision	0.7228	->	0.5635	(-0.1593)
recall	0.5393	->	0.8246	(+0.2853)
specificity	0.9621	->	0.8829	(-0.0792)
f1	0.6177	->	0.6695	(+0.0518)

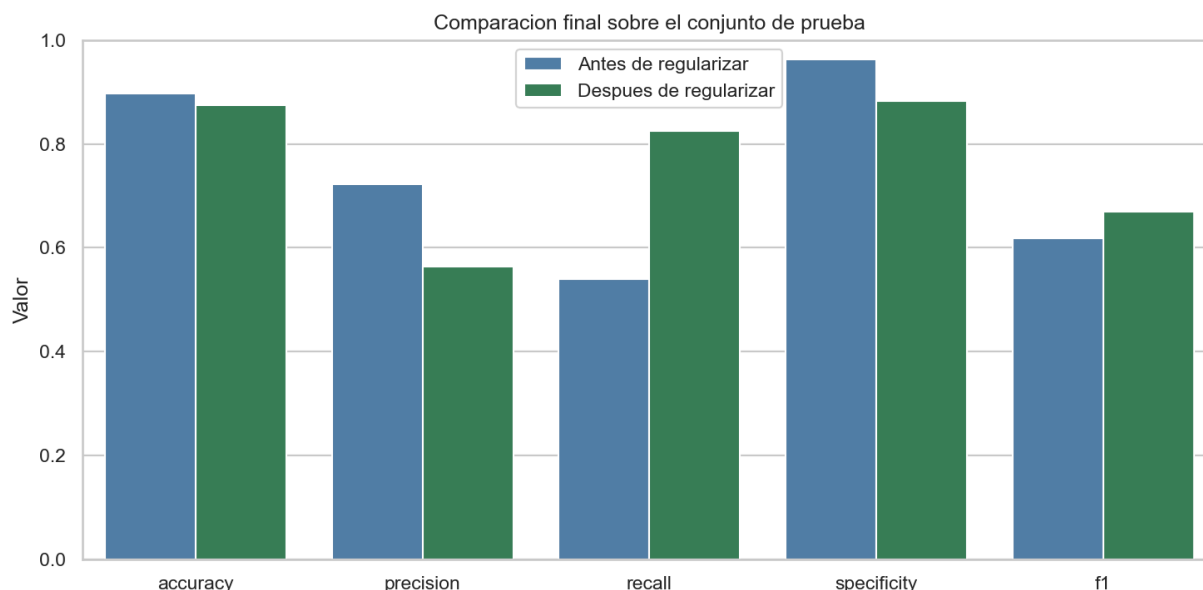


Figura 5. Comparación de las cinco métricas, antes y después de regularizar, conjunto de prueba.

El F1 mejoró de 0.6177 a 0.6695 (+0.0518), confirmando en el conjunto de prueba lo que ya se veía en validación. El cambio más grande está en recall, que subió de 0.5393 a 0.8246: el modelo regularizado identifica correctamente 315 de los 382 compradores reales del conjunto de prueba (82%), contra 206 (54%) del modelo base. Esto es consecuencia directa de `class_weight='balanced'`, que penaliza más los errores sobre la clase minoritaria durante el entrenamiento. El costo es una caída en precision (de 0.7228 a 0.5635) y en accuracy (de 0.8966 a 0.8739): el modelo ahora genera más falsos positivos (244 contra 79 antes). Retomando el mismo problema de negocio planteado en la parte 2 (decidir a qué usuarios mostrar una promoción durante la sesión), donde un falso negativo cuesta una venta perdida y un falso positivo solo cuesta mostrar una promoción de más, este intercambio es favorable: se sacrifican accuracy y precision para casi duplicar la capacidad del modelo de detectar compradores reales.

Análisis y Conclusiones

El análisis confirma con evidencia cuantitativa lo que las partes anteriores ya sugerían de forma indirecta: el Random Forest sin límite de profundidad memoriza el conjunto de entrenamiento (bias bajo, varianza alta, overfit), y limitar su capacidad —junto con compensar el desbalance de clases— produce un modelo que generaliza mejor, al costo de un F1 de entrenamiento más modesto. El hallazgo más importante de la búsqueda de hiperparámetros no fue `max_depth` ni `n_estimators`, sino `class_weight='balanced'`: casi todas las combinaciones en el top del GridSearchCV lo usan, y es el hiperparámetro que más mueve la aguja del recall. Esto tiene sentido con el desbalance de clases documentado desde la parte 1 (84.5% vs 15.5%): sin compensarlo, el modelo tiene pocos incentivos durante el entrenamiento para aprender bien la clase minoritaria, sin importar qué tan bien regularizada esté su profundidad.

Vale la pena ser honesto sobre una limitación de este análisis: los umbrales usados para clasificar bias y varianza como bajo/medio/alto (sección de metodología) son un criterio razonado pero no una convención universal de la literatura, y el diagnóstico de “fit” del modelo regularizado todavía tiene una brecha train-validación de casi 0.09, que sin ser alta, tampoco es despreciable. Un ajuste más fino, o el uso de validación cruzada en lugar de un único conjunto de validación, podría dar un diagnóstico aún más robusto.

Como trabajo futuro: explorar el punto óptimo entre recall y precision ajustando el umbral de decisión de `predict_proba` en vez de solo `class_weight`, y repetir este mismo análisis de bias-varianza con validación cruzada de k folds para confirmar que el diagnóstico no depende de la partición particular de datos usada aquí.

Referencias

Sakar, C. & Kastro, Y. (2018). Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository. <https://doi.org/10.24432/C5F88Q>

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Ramírez Uresti, J. A. sf. Tema 4: Evaluación y Diagnóstico de Problemas en los Modelos. Material del curso Inteligencia Artificial Avanzada para la Ciencia de Datos, Módulo 2. Instituto Tecnológico de Estudios Superiores de Monterrey.

Código fuente, gráficas y datos:

https://github.com/A01801224/Mod2_Portafolio_Implementacion